

Content

- Operating Systems:
 - Introduction to different types of operating
 - Real Time Operating Systems,
 - System Components,
 - OS services,
 - System structure- Layered Approach.
- Process Management:
 - Process Concept- Process states,
 - Process control block,
 - Threads,
 - **Inter-process Communication,**
 - **Concurrency & Synchronization,**
 - Process Scheduling:
 - Types of process schedulers,
 - Types of scheduling: Preemptive,
 - Non preemptive.
 - Scheduling algorithms: FCFS, SJF, RR, Priority,

Deadlocks:

Methods of handling deadlocks,
Deadlock prevention,
avoidance and detection,
Recovery from deadlocks.

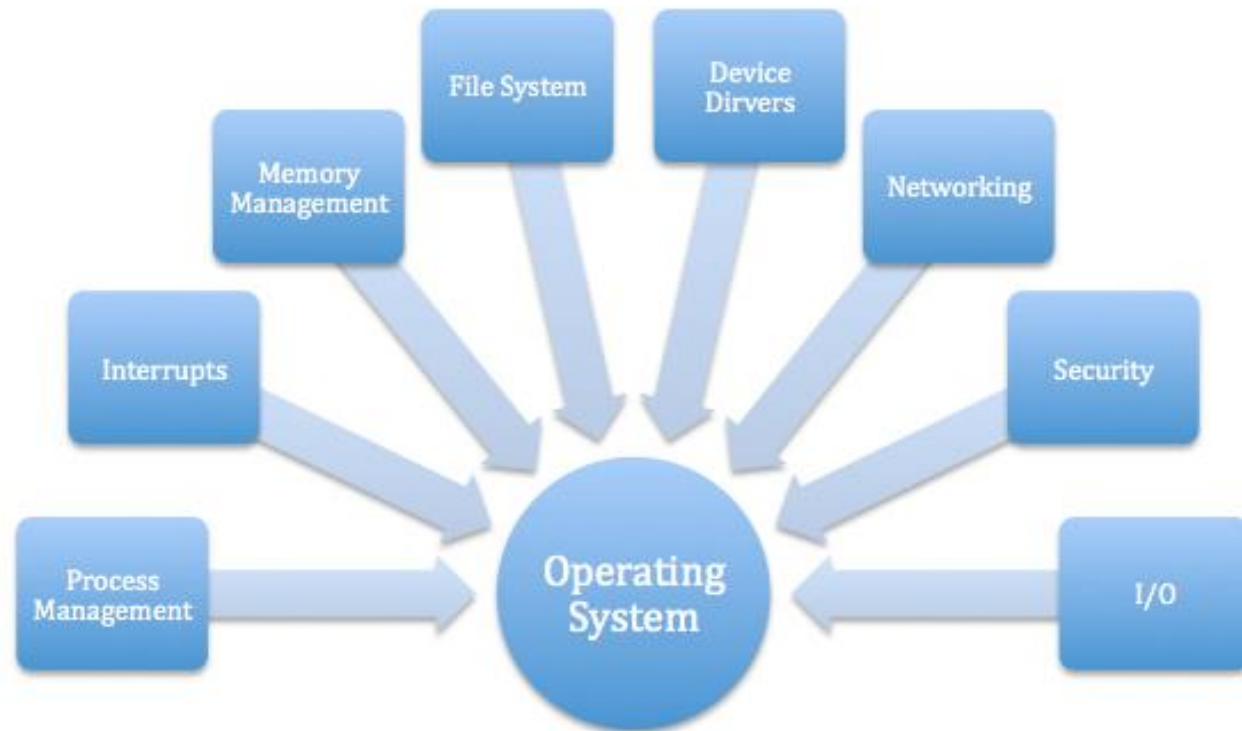
Case Study: Process Management in multi-cores OS.



Operating System



- A program that controls the execution of application programs
- An interface between applications and hardware

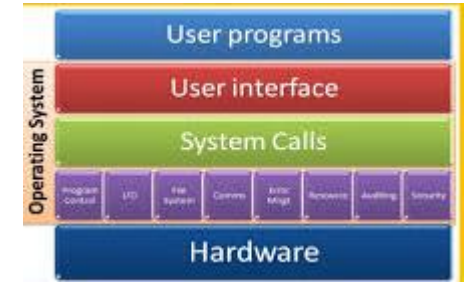




Operating System Objectives

- Convenience
 - Makes the computer more convenient to use
- Efficiency
 - Allows computer system resources to be used in an efficient manner
- Ability to evolve
 - Permit effective development, testing, and introduction of new system functions without interfering with service

Operating System :: Services



- Program development
 - Editors and debuggers
- Program execution
- Access to I/O devices
- Controlled access to files
- System access
- Error detection and response
 - Internal and external hardware errors
 - Memory error
 - Device failure
 - Software errors
 - Arithmetic overflow
 - Access forbidden memory locations
 - Operating system cannot grant request of application

Accounting

Collect usage statistics

Monitor performance

Used to anticipate future enhancements

Used for billing purposes

Responsible for managing resources

Functions same way as ordinary computer software

It is program that is executed

Operating system relinquishes control of the processor

Types of operating Systems

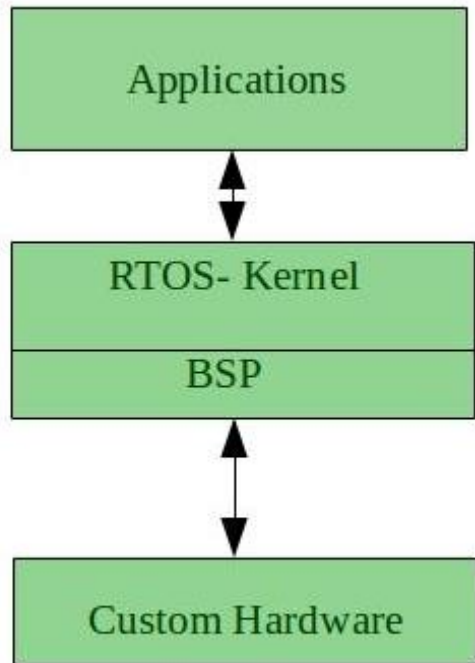
Distinguished by the response time and how data is entered into the system

- Single user
- Multi user
- Multitasking
- Multi processing
- Embedded
- Real time

Real Time Operating System

- These types of OSs serve the real-time systems. The time interval required to process and respond to inputs is very small. This time interval is called **response time**.
- **Real-time systems** are used when there are time requirements that are very strict like missile systems, air traffic control systems, robots etc.
- **Two types of Real-Time Operating System which are as follows:**
- **Hard Real-Time Systems:**
These OSs are meant for the applications where time constraints are very strict and even the shortest possible delay is not acceptable. These systems are built for saving life like automatic parachutes or air bags which are required to be readily available in case of any accident. Virtual memory is almost never found in these systems.
- **Soft Real-Time Systems:**
These OSs are for applications where the time-constraint is less strict.

RTOS Details:



Advantages of RTOS:

- **Maximum Consumption:** Maximum utilization of devices and system, thus more output from all the resources
- **Task Shifting:** Time assigned for shifting tasks in these systems are very less. For example in older systems it takes about 10 micro seconds in shifting one task to another and in latest systems it takes 3 micro seconds.
- **Focus on Application:** Focus on running applications and less importance to applications which are in queue.
- **Real time operating system in embedded system:** Since size of programs are small, RTOS can also be used in embedded systems like in transport and others.
- **Error Free:** These types of systems are error free.
- **Memory Allocation:** Memory allocation is best managed in these type of systems.

Disadvantages of RTOS:

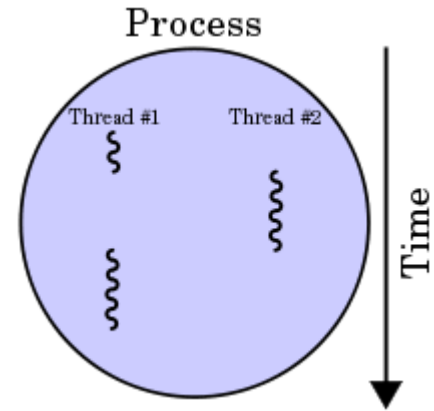
- **Limited Tasks:** Very few task run at the same time and their concentration is very less on few applications to avoid errors.
- **Use heavy system resources:** Sometimes the system resources are not so good and they are expensive as well.
- **Complex Algorithms:** The algorithms are very complex and difficult for the designer to write on.
- **Device driver and interrupt signals:** It needs specific device drivers and interrupt signals to response earliest to interrupts.
- **Thread Priority:** It is not good to set thread priority as these systems are very less pron to switching tasks.

Examples of Real-Time Operating Systems

are: Scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, air traffic control systems, etc.

Process Management:

- Process Concept- Process states,
- Process control block,
- Threads,
- **Inter-process Communication,**
- **Concurrency & Synchronization,**
- Process Scheduling:
- Types of process schedulers,
- Types of scheduling: Preemptive,
- Non preemptive.
- Scheduling algorithms: FCFS, SJF, RR, Priority



What is Process ???

- A process is a program during execution.
 - Program is a static file
 - Process is execution program.
- A process is the basic unit of execution in an operating system
 - Each process has a number, its process identifier (pid).
- Process requires following components :
 - object program
 - Data
 - Resources
 - Status

Difference between

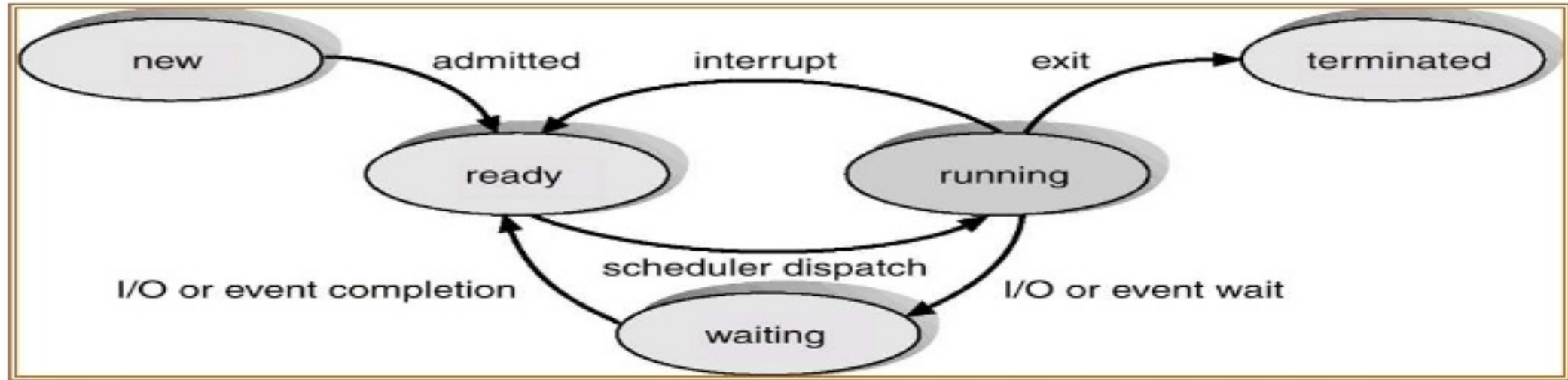
PROGRAM

- It is static entity made up of program statements
- Program defines a task or job for OS
- It is Considered to be Passive entity.
- Programs do not compete for resources like CPU, Memory

PROCESS

- Process is dynamic entity that is program in execution
- 2 or more processes could be executing the same program, each using their own data.
- It is active entity.
- Process do compete for resources like CPU, Memory

Process States

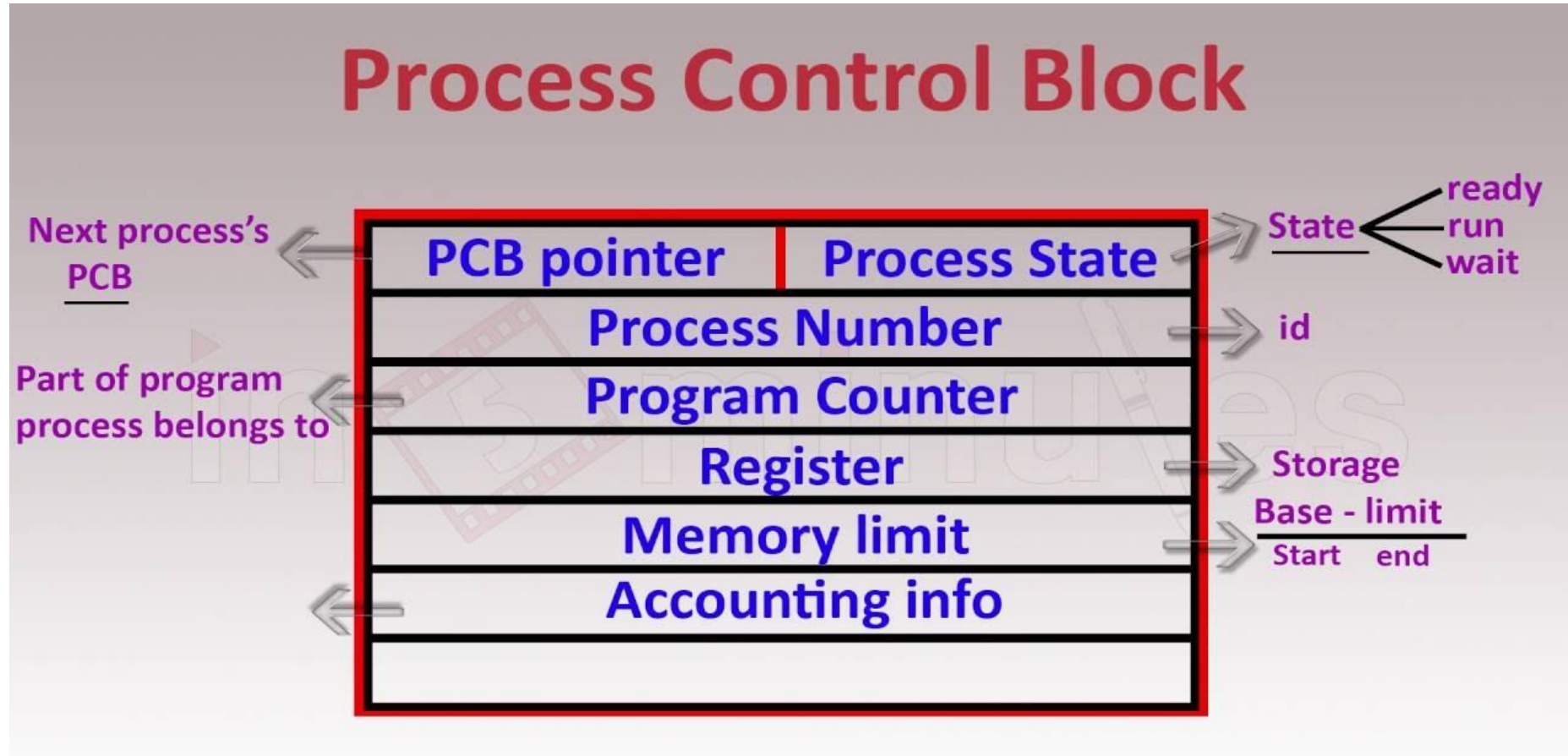


Process States

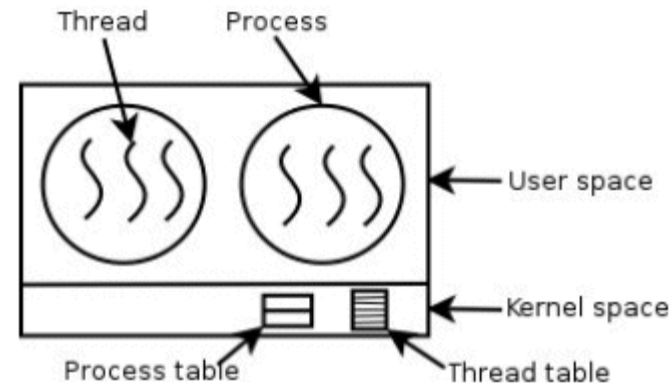
- New- The process is being created.
- Running- Instructions are being executed.
- Waiting- The process is waiting for some event to occur.
- Ready- The process is waiting to be assigned to a processor.
- Terminated- The process has finished execution.

Process Control Block- contains all information associated with a specific process like process state, program counter, CPU registers and info regarding CPU scheduling algorithms, memory, I/O and accounting.

PCB

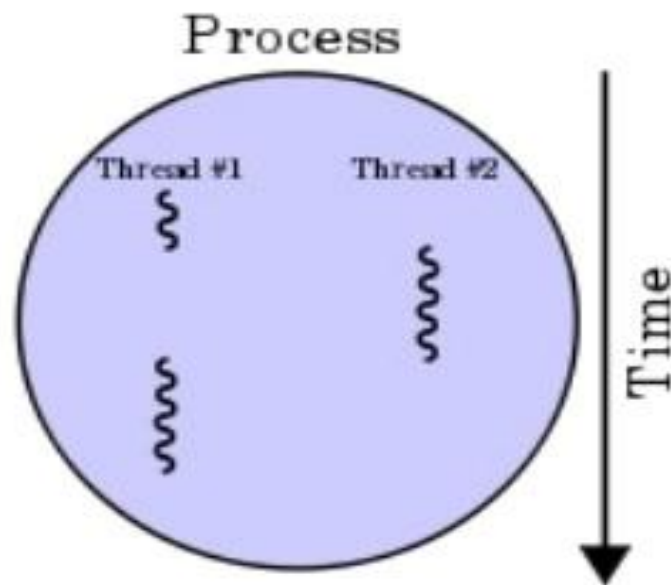


[INTRODUCTION



- A thread is contained inside a process and different threads in the same process share some resources (most commonly **memory**), while different **processes** do not.

Process and threads



What is thread?

- A thread is a **flow of execution** through the process code, with its own program counter, system registers and stack.
- A thread is also called a **light weight process**. Threads provide a way to improve application performance through parallelism.
- Each thread belongs to exactly **one process and no thread can exist outside** a process.
- Each thread represents a **separate flow of control**.
- Threads have been successfully used in implementing network servers and web server. They also provide a suitable foundation for parallel execution of applications on shared memory multiprocessors

Difference between process & thread

Process

- Process is heavy weight or resource intensive.
- Process switching needs **interaction** with OS.
- In multiple process environment each process executes the **same code but has its own memory & file resources**.
- If one process is **blocked** then no other process can execute until the first process is unblocked.
- Multiple process without using threads use more resources.
- In multiple processes each process operates independently of other.

Thread

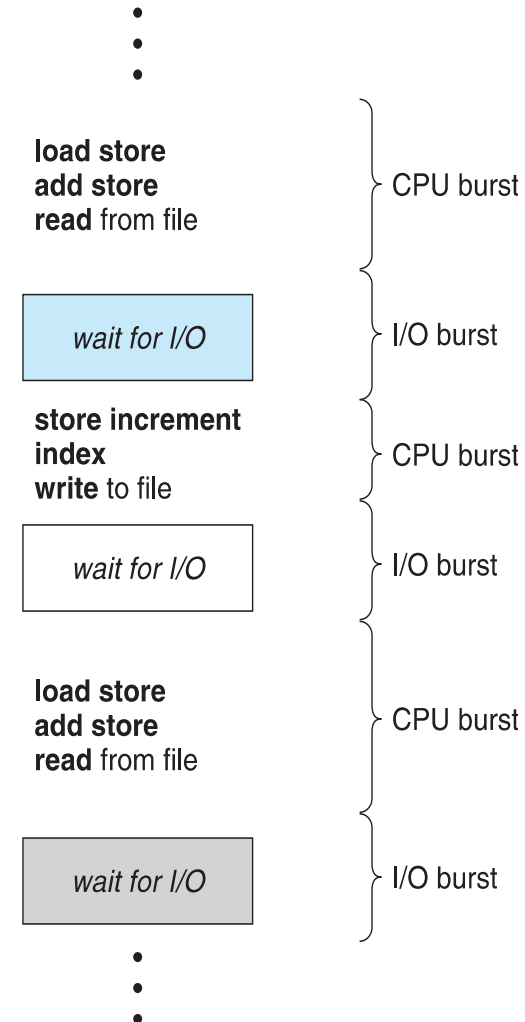
- Thread is light weight taking lesser resources than a process.
- Thread switching does not interact with OS
- All threads can **share same set of open files, child processes**
- While one thread is blocked and waiting , second thread in the same task can run.
- Multiple threads uses fewer resources.
- One thread can read , write or change another thread's data.

CPU SCHEDULING

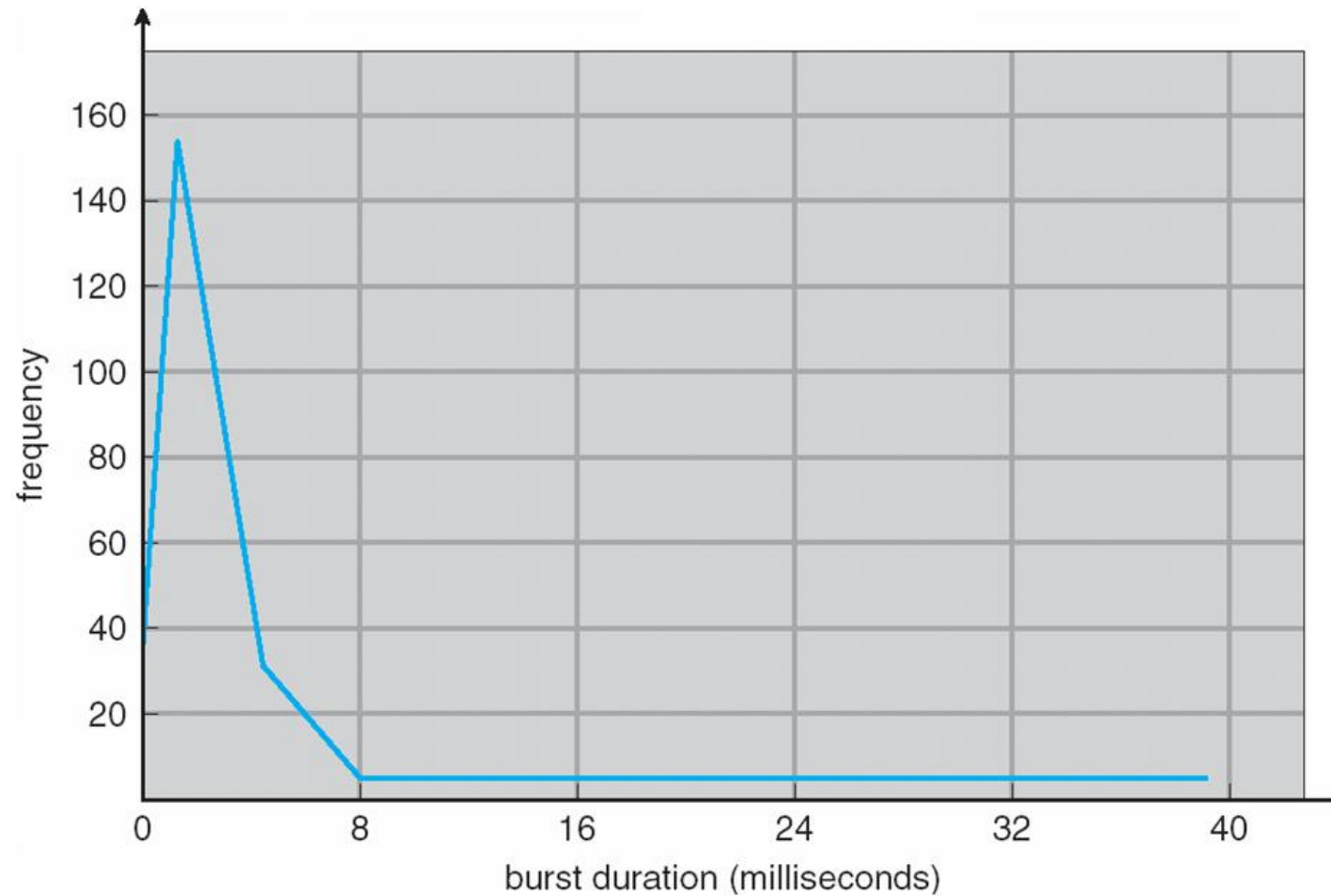
- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Thread Scheduling
- Multiple-Processor Scheduling
- Real-Time CPU Scheduling
- Operating Systems Examples
- Algorithm Evaluation

BASIC concepts

- Maximum CPU utilization obtained with multiprogramming
- CPU–I/O Burst Cycle – Process execution consists of a **cycle** of CPU execution and I/O wait
- **CPU burst** followed by **I/O burst**
- CPU burst distribution is of main concern



CPU – I/O Burst Cycle



CPU Scheduler

- When CPU becomes idle , the OS must select one of the processes in ready queue to be executed. The selection process is carried out by a short-term scheduler.
- **Short-term scheduler** selects from among the processes in ready queue, and allocates the CPU to one of them
 - Queue may be ordered in various ways
- CPU scheduling decisions may take place when a process:
 1. Switches from running to waiting state
 2. Switches from running to ready state
 3. Switches from waiting to ready
 4. Terminates
- Scheduling under 1 and 4 is **nonpreemptive**
- All other scheduling is **preemptive**
 - Consider access to shared data
 - Consider preemption while in kernel mode
 - Consider interrupts occurring during crucial OS activities

SCHEDULERS

- Long-term scheduler (or job scheduler) – selects which processes should be brought into the ready queue.
- Short-term scheduler (or CPU scheduler) – selects which process should be executed next and allocates CPU.

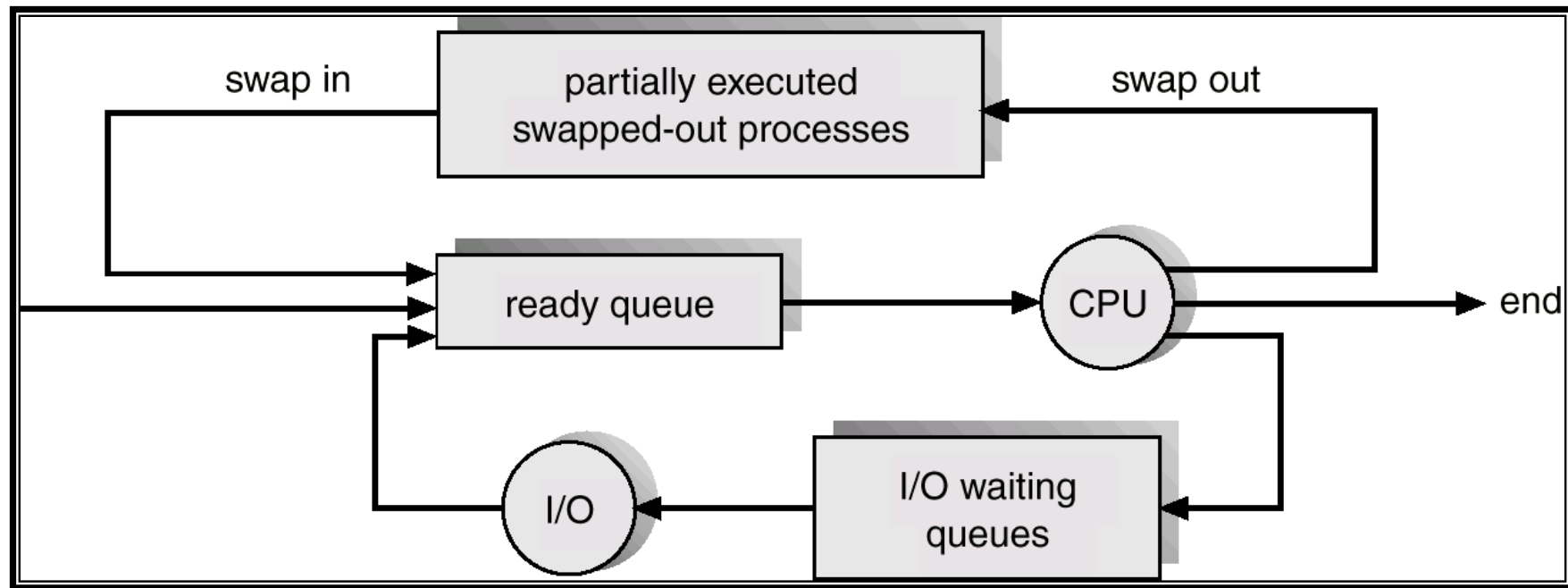
SCHEDULERS

- **Medium – Term Scheduler :**

The key idea is to remove process from memory & thus reduce the degree of multiprogramming .

- And later when required can be swapped into memory back.
- Swapping is conducted by Medium term scheduler

SCHEDULERS



Queuing – diagram representation of process scheduling

SCHEDULERS

- Processes can be described as either:
 - *I/O-bound process* – spends more time **doing I/O** than computations.
 - Ready Queue will be almost empty
 - *CPU-bound process* – spends more time **doing computations**;
 - Waiting Queue will be empty.

CONTEXT SWITCH

What is a process context?

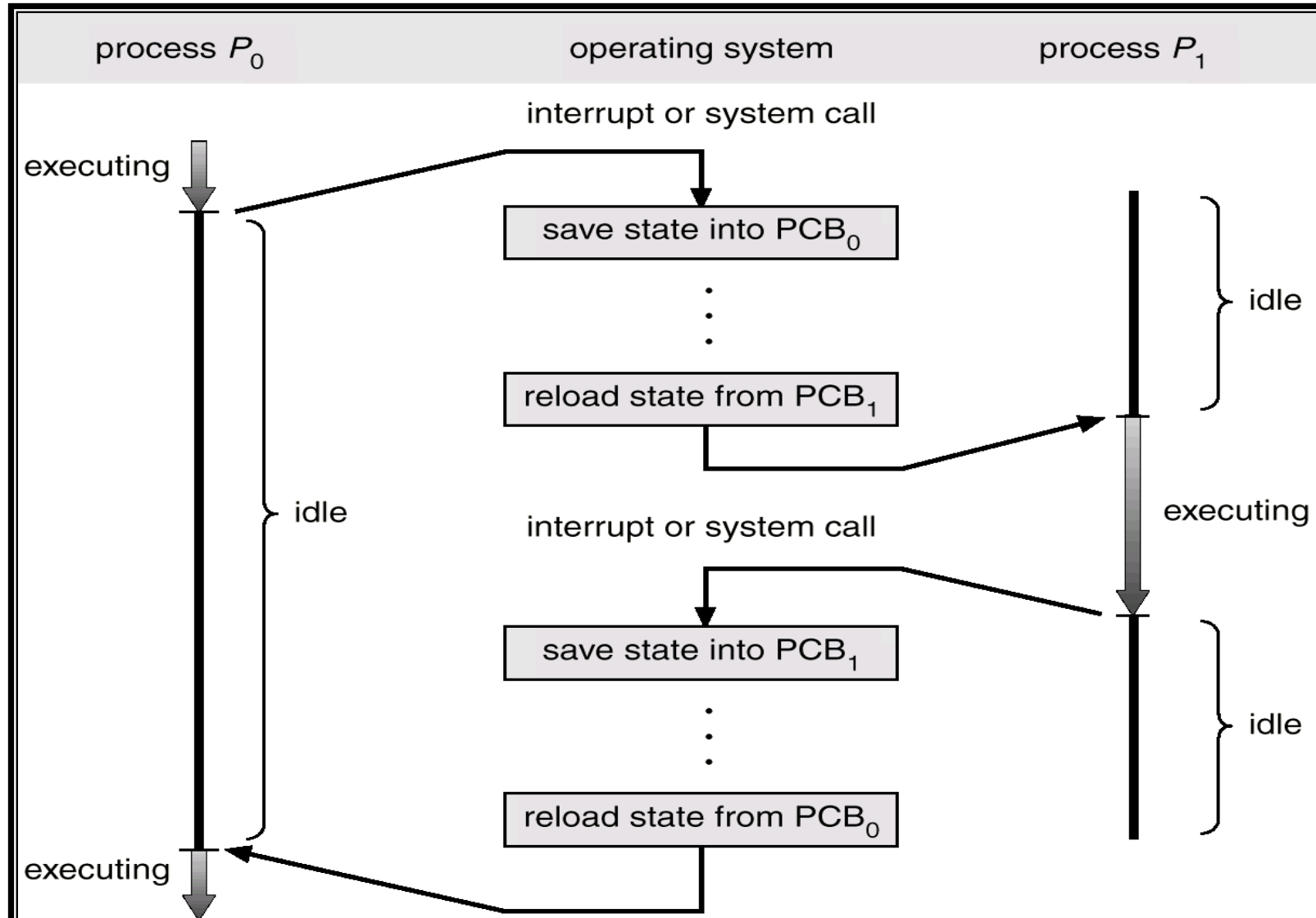
The *context of a* process includes the values of CPU registers, the process state, the program counter, and other memory/file management information.

What is a context switch?

After the CPU scheduler selects a process (from the *ready queue*) and before allocates CPU to it, the CPU scheduler must

1. save the *context of the currently running process*,
2. put it into a queue,
3. load the *context of the selected process*, and
4. let it run.

CONTEXT SWITCH



Dispatcher

- Dispatcher is a module that gives control of the CPU to the process selected by the short-term scheduler;
- this function involves:
 - switching context
 - switching to user mode
 - jumping to the proper location in the user program to restart that program
- **Dispatch latency** – time it takes for the dispatcher to stop one process and start another running

Scheduling Criteria

CPU utilization – keep the CPU as busy as possible (from 0% to 100%)

Throughput – # of processes that complete their execution per time unit

Turnaround time – amount of time to execute a particular Process

Waiting time – amount of time a process has been waiting in the ready queue

Response time – amount of time it takes from when a request was submitted until the first response is produced

Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min Response time

Scheduling Algorithms

- **First Come First Serve Scheduling**
- **Shortest Job First Scheduling**
- **Priority Scheduling**
- **Round-Robin Scheduling**
- **Multilevel Queue Scheduling**
- **Multilevel Feedback-Queue Scheduling**

First Come First Serve Scheduling (FCFS)

<u>Process</u>	<u>Burst time</u>
P1	24
P2	3
P2	3

Suppose that the processes arrive in the order: P_1 , P_2 , P_3
The Gantt Chart for the schedule is:



First Come First Serve Scheduling

- The average of waiting time in this policy is usually quite long
- Waiting time for P1=0; P2=24; P3=27
- Average waiting time= $(0+24+27)/3=17$

First Come First Serve Scheduling

- Consider if we have a CPU-bound process and many I/O-bound processes
- There is a **convoy effect** as all the other processes waiting for one of the big process to get off the CPU
- FCFS scheduling algorithm is non-preemptive

Short job first scheduling (SJF)

- This algorithm associates with each process the length of the processes' next CPU burst
- If there is a tie, FCFS is used
- In other words, this algorithm can be also regard as shortest-next-cpu-burst algorithm

Short job first scheduling

- SJF is optimal – gives minimum average waiting time for a given set of processes

Example

<u>Processes</u>	<u>Burst time</u>
------------------	-------------------

P1	6
----	---

P2	8
----	---

P3	7
----	---

P4	3
----	---

FCFS average waiting time: $(0+6+14+21)/4=10.25$

SJF average waiting time: $(3+16+9+0)/4=7$

Short job first scheduling

Two schemes:

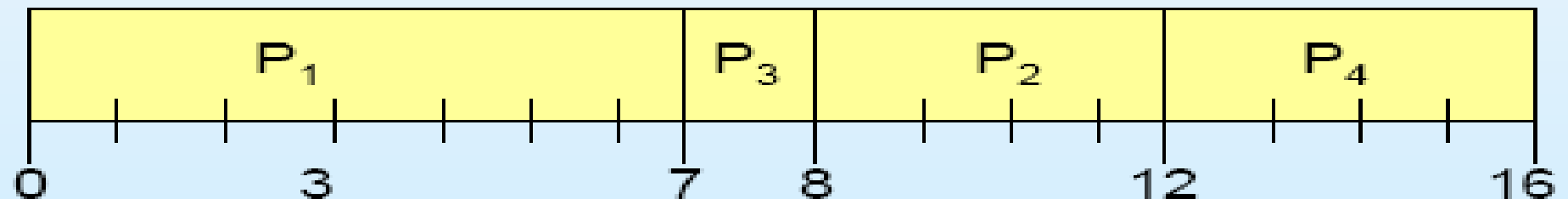
Non-preemptive – once CPU given to the process it cannot be preempted until completes its CPU burst

Preemptive – if a new process arrives with CPU burst length less than remaining time of current executing process, preempt. This scheme is known as the Shortest-Remaining-Time-First (SRTF)

Short job first scheduling- Non-preemptive

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

SJF (non-preemptive)

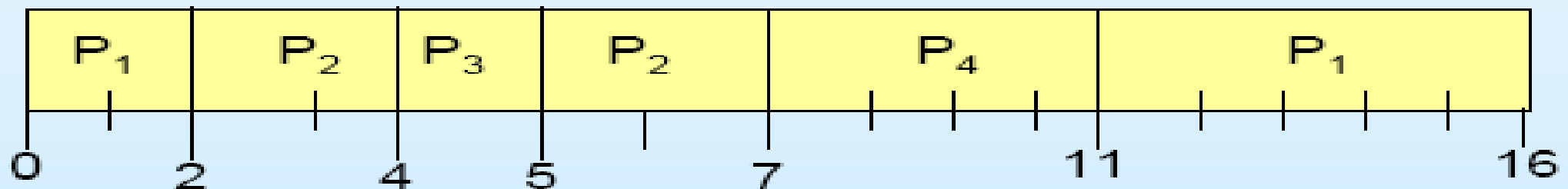


$$\text{Average waiting time} = (0 + 6 + 3 + 7)/4 = 4$$

Short job first scheduling- Preemptive

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

SJF (preemptive)



$$\text{Average waiting time} = (9 + 1 + 0 + 2)/4 = 3$$

Priority Scheduling

A priority number (integer) is associated with each process

The CPU is allocated to the process with the highest priority
(smallest integer $\equiv$ highest priority)

- Preemptive
- Non-preemptive

SJF is a special priority scheduling where priority is the predicted next CPU burst time, so that it can decide the priority

Priority Scheduling

<u>Processes</u>	<u>Burst time</u>	<u>Priority</u>	<u>Arrival time</u>
P1	10	3	
P2	1	1	
P3	2	4	
P4	1	5	
P5	5	2	

The average waiting time= $(6+0+16+18+1)/5=8.2$

Priority Scheduling

<u>Processes</u>	<u>Burst time</u>	<u>Priority</u>	<u>Arrival time</u>
P1	10	3	0.0
P2	1	1	1.0
P3	2	4	2.0
P4	1	5	3.0
P5	5	2	4.0

Gantt chart for both preemptive and non-preemptive, also waiting time

Priority Scheduling

Problem : Starvation – low priority processes may never execute

Solution : Aging – as time progresses increase the priority of the process

Round-Robin Scheduling

- The Round-Robin is designed especially for time sharing systems.
- It is similar FCFS but add preemption concept
- A small unit of time, called **time quantum**, is defined

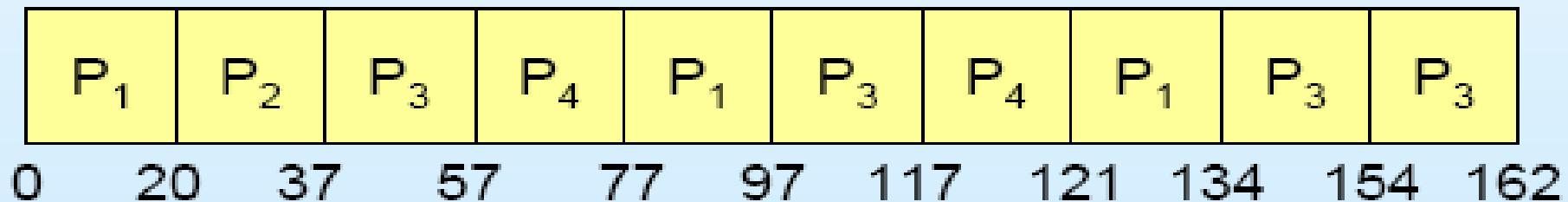
Round-Robin Scheduling

- Each process gets a small unit of CPU time (*time quantum*), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.

Round-Robin Scheduling

<u>Process</u>	<u>Burst Time</u>
P_1	53
P_2	17
P_3	68
P_4	24

The Gantt chart is:



Round-Robin Scheduling

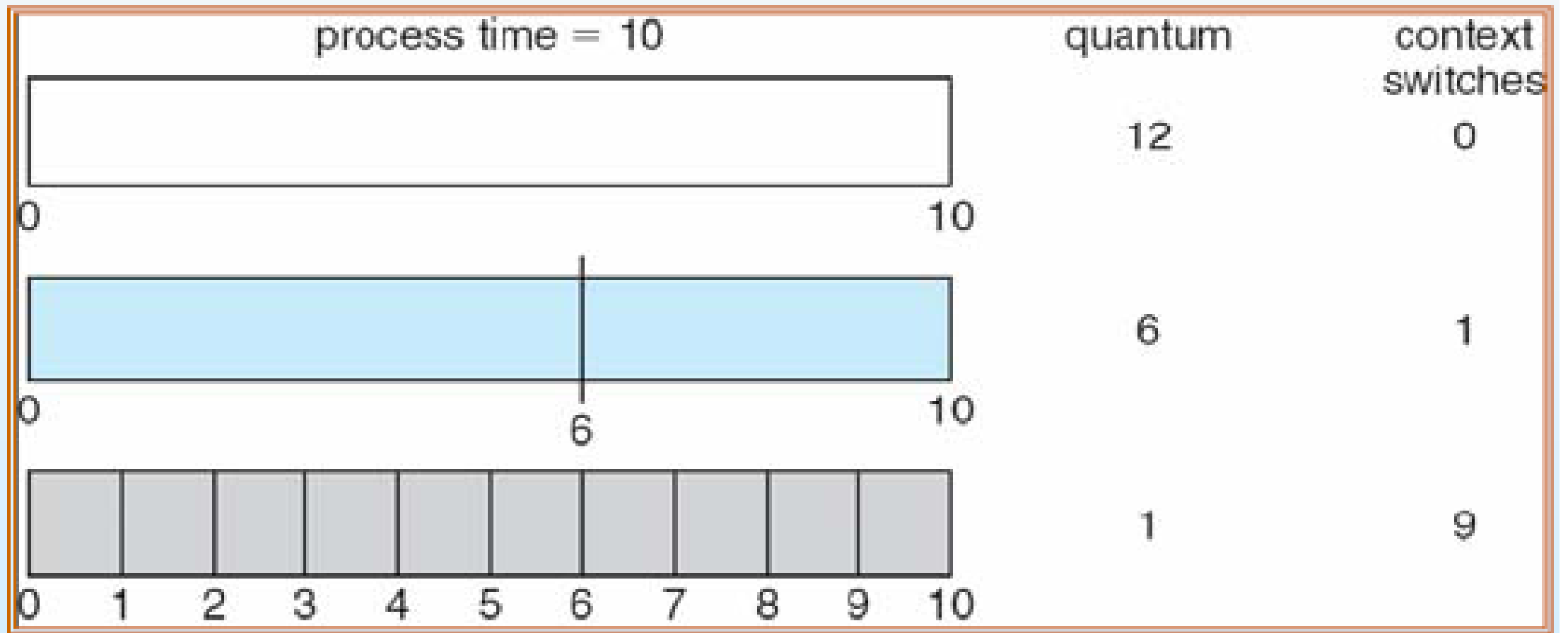
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units.

Round-Robin Scheduling

Performance

- q large $\Rightarrow$ FIFO
- q small $\Rightarrow q$ must be large with respect to context switch, otherwise overhead is too high
- Typically, higher average turnaround than SJF, but better *response*

Round-Robin Scheduling



Multilevel Queue

Ready queue is partitioned into separate queues:

- foreground (interactive)
- background (batch)

Each queue has its own scheduling algorithm

foreground – RR

background – FCFS

Multilevel Queue example

- Foreground P1 53 (RR interval:20)

P2 17

P3 42

- Background P4 30 (FCFS)

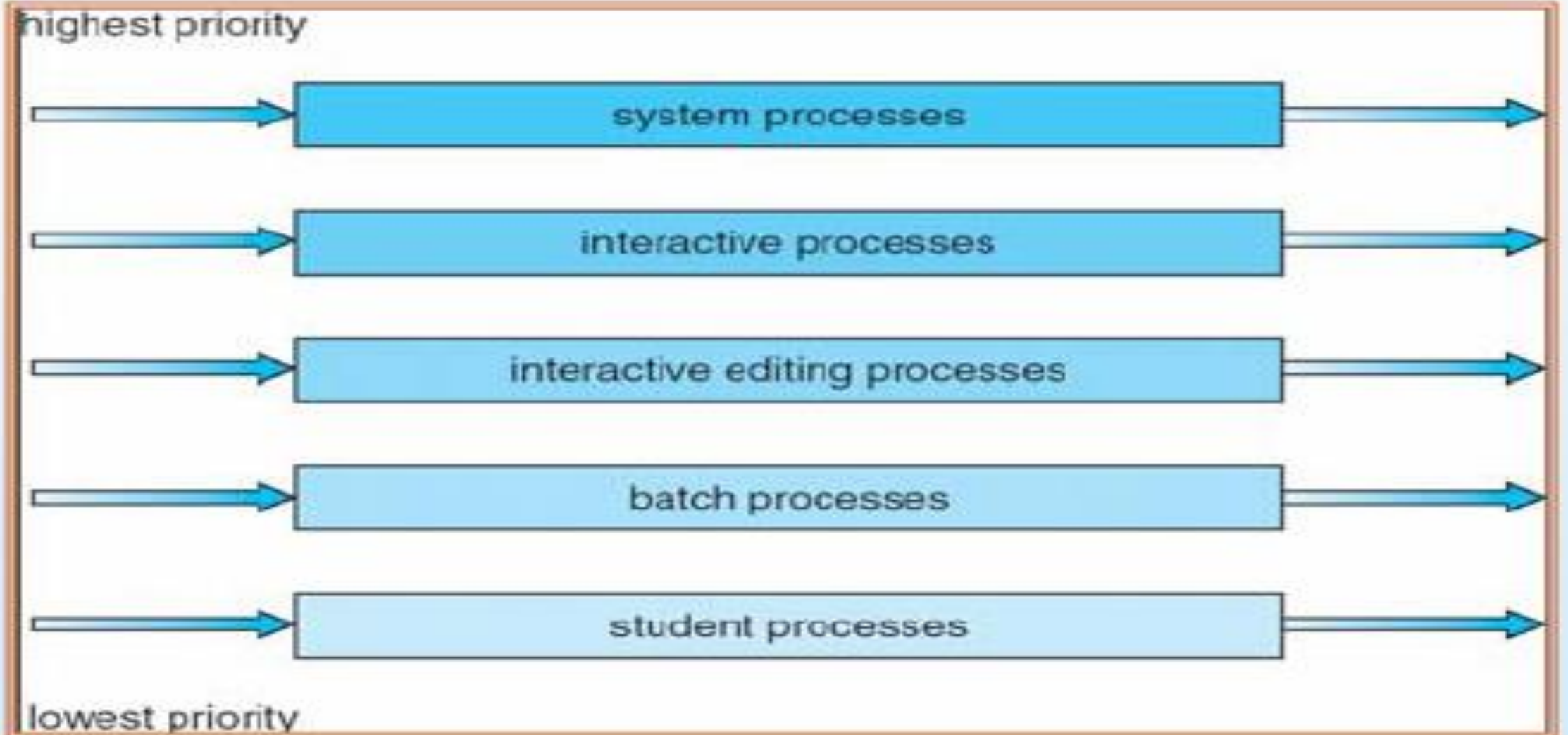
P5 20

Multilevel Queue

Scheduling must be done between the queues

- Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
- Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR

Multilevel Queue



Multilevel Feedback Queue

Three queues:

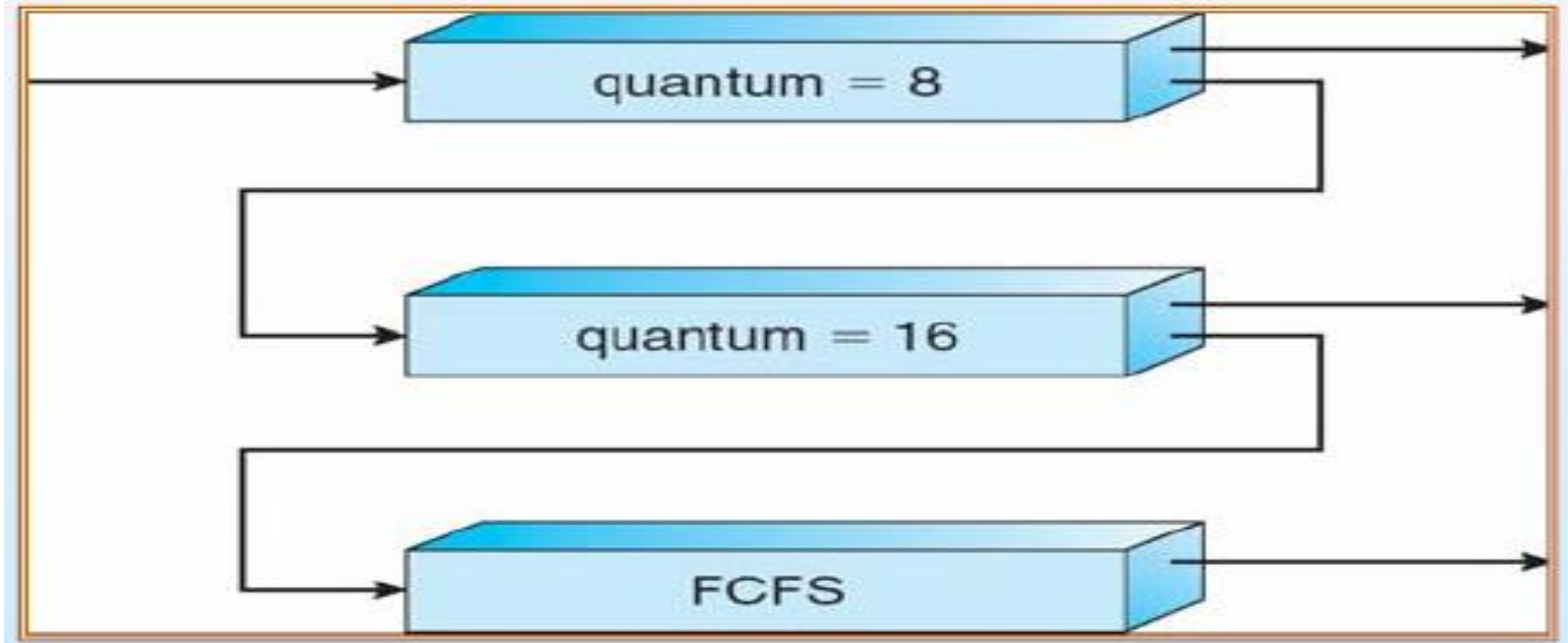
- ***Q0* – RR with time quantum 8 milliseconds**
- ***Q1* – RR time quantum 16 milliseconds**
- ***Q2* – FCFS**

Scheduling

A new job enters queue *Q0* which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue *Q1*.

At *Q1* job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue *Q2*.

Multilevel Feedback Queue



Multilevel Feedback Queue

- P1 40
- P2 35
- P3 15

Multiple-Processor Scheduling

- We concentrate on systems in which the processors are identical (homogeneous)
- **Asymmetric multiprocessing** (by one master) is simple because only one processor access the system data structures.
- **Symmetric multiprocessing**, each processor is self-scheduling. Each processor may have their own ready queue.

Load balancing

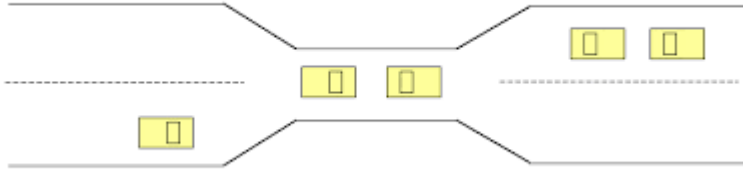
- On symmetric multiprocessing systems, it is important to keep the workload balanced among all processors to fully utilized the benefits of having more than one CPU
- There are two general approached to load balancing: **Push Migration** and **Pull Migration**

Symmetric Multithreading

- An alternative strategy for symmetric multithreading is to provide multiple logical processors (rather than physical)
- It's called **hyperthreading technology** on Intel processors

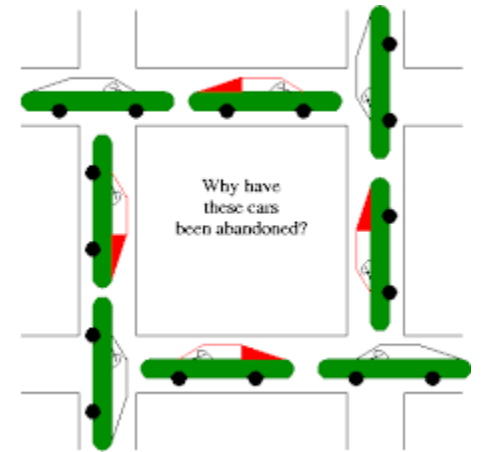
Symmetric Multithreading

- The idea behind it is to create multiple logical processors on the same physical processor (sounds like two threads)
- But it is not software provide the feature, but hardware
- Each logical processor has its own architecture state, each logical processor is responsible for its own interrupt handling.



Deadlock

- Prevention
- Detection
- Recovery



The Deadlock problem

- In a computer system deadlocks arise when members of a group of processes which hold resources are blocked indefinitely from access to resources held by other processes within the group.

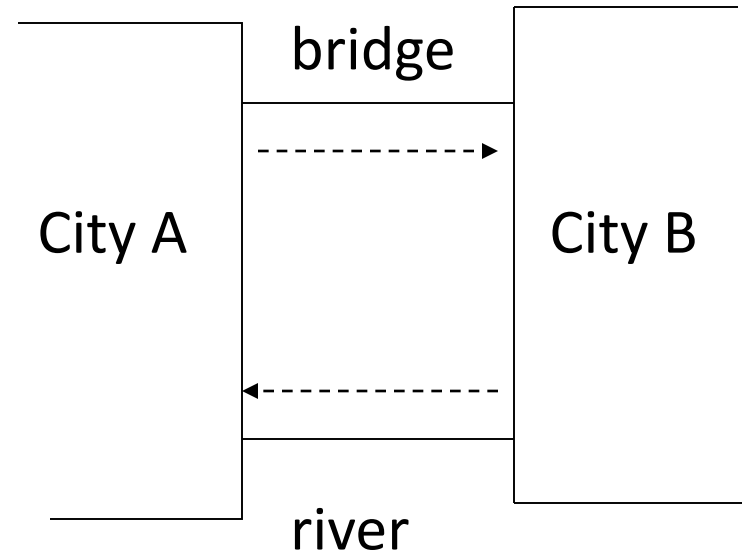
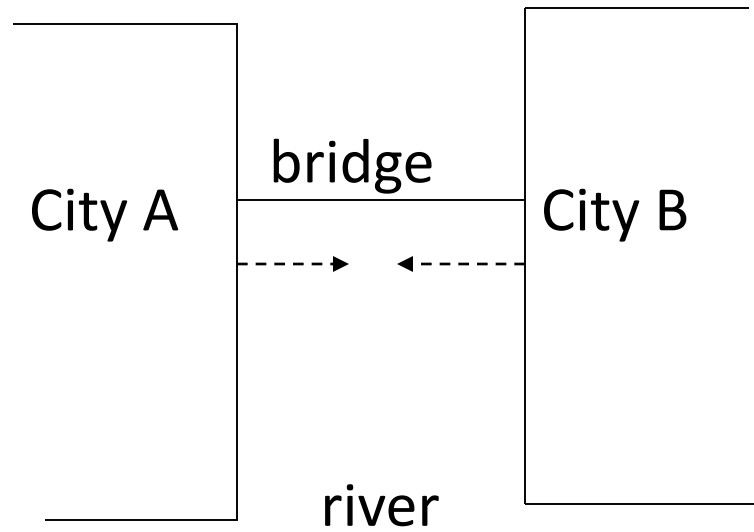
Deadlock example

- P_i requests one I/O controller and the system allocates one.
- P_j requests one I/O controller and again the system allocates one.
- P_i wants another I/O controller but has to wait since the system ran out of I/O controllers.
- P_j wants another I/O controller and waits.

Conditions for deadlocks

- *Mutual exclusion*. No resource can be shared by more than one process at a time.
- *Hold and wait*. There must exist a process that is holding at least one resource and is waiting to acquire additional resources that are currently being held by other processes.
- *No preemption*. A resource cannot be preempted.
- *Circular wait*. There is a cycle in the wait-for graph.

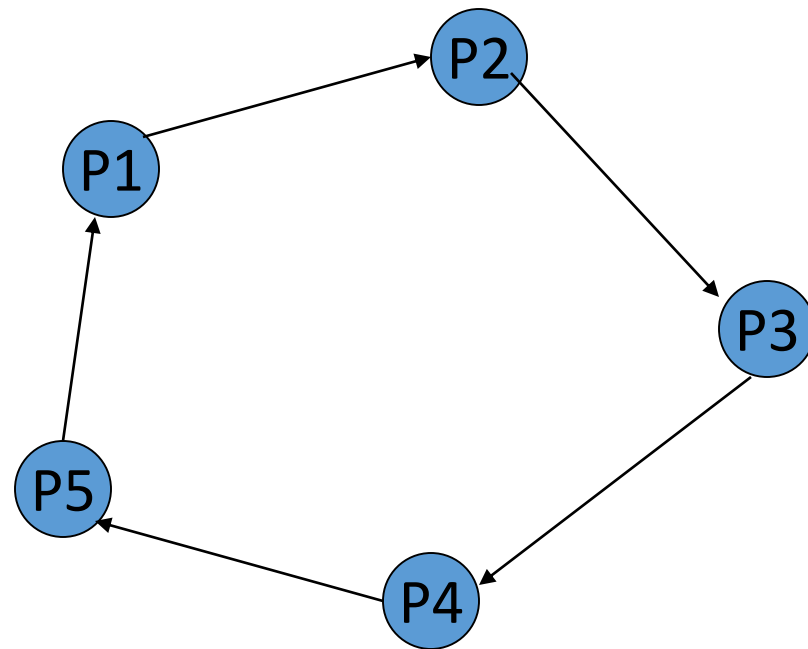
An example



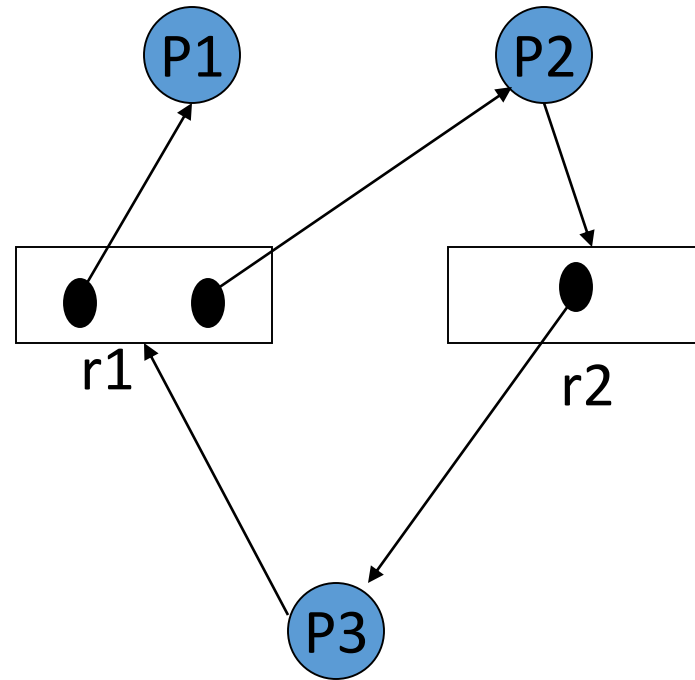
Graph-theoretic models

- Wait-for graph.
- Resource-allocation graph.

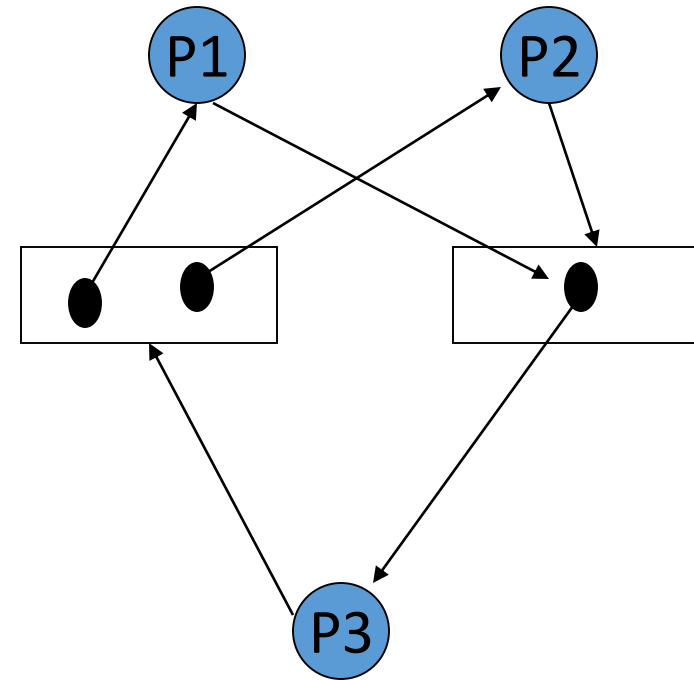
Wait-for graph



Resource allocation graph



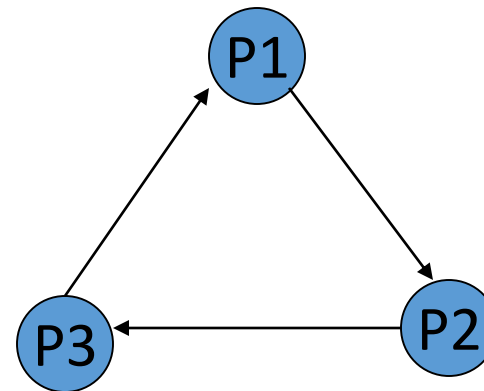
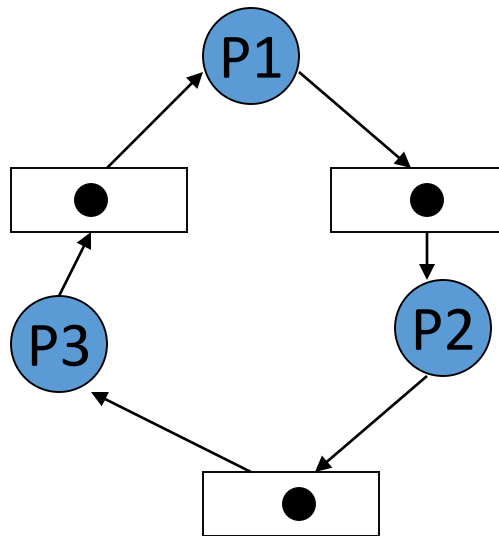
Resource allocation graph
Without deadlock



With deadlock

Wait-for graph and Resource-allocation graph conversion

- Any resource allocation graph with a single copy of resources can be transferred to a wait-for graph.



Strategies for handling deadlocks

- Deadlock prevention. Prevents deadlocks by restraining requests made to ensure that at least one of the four deadlock conditions cannot occur.
- Deadlock avoidance. Dynamically grants a resource to a process if the resulting state is safe. A state is safe if there is at least one execution sequence that allows all processes to run to completion.
- Deadlock detection and recovery. Allows deadlocks to form; then finds and breaks them.

Two types of deadlocks

- Resource deadlock: uses AND condition.

AND condition: a process that requires resources for execution can proceed when it has acquired all those resources.

- Communication deadlock: uses OR condition.

OR condition: a process that requires resources for execution can proceed when it has acquired at least one of those resources.

- P-out-of-Q condition which means that a process simultaneously requests Q resources and remains blocked until it is granted any P of those resources.
- AND-OR model, which may specify any combination of AND and OR models.

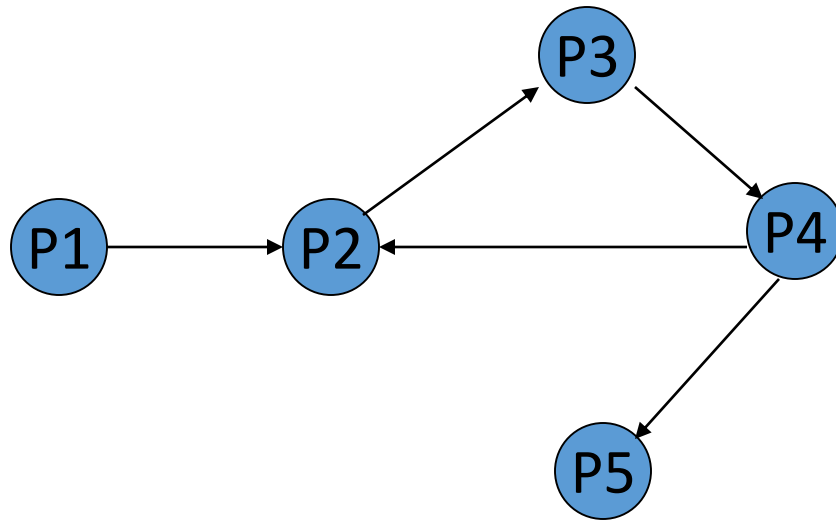
E.g. a AND (b OR c).

Deadlock conditions

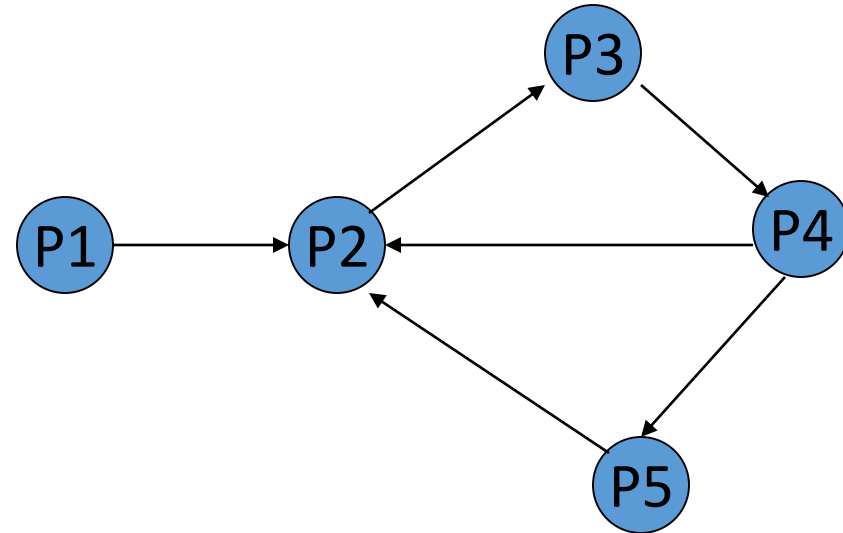
- The condition for deadlock in a system using the AND condition is the existence of a *cycle*.
- The condition for deadlock in a system using the OR condition is the existence of a *knot*.

A knot (K) consists of a set of nodes such that for every node a in K, all nodes in K and only the nodes in K are reachable from node a .

Example: OR condition



No deadlock



Deadlock

Deadlock Prevention

- 1. A process acquires all the needed resources simultaneously before it begins its execution, therefore breaking the hold and wait condition.
- E.g. In the dining philosophers' problem, each philosopher is required to pick up both forks at the same time. If he fails, he has to release the fork(s) (if any) he has acquired.
- Drawback: over-cautious.

- 2. All resources are assigned unique numbers. A process may request a resource with a unique number l only if it is not holding a resource with a number less than or equal to l and therefore breaking the circular wait condition.
- E.g. In the dining philosophers problem, each philosopher is required to pick a fork that has a larger id than the one he currently holds. That is, philosopher P_5 needs to pick up fork F_5 and then F_1 ; the other philosopher P_i should pick up fork F_i followed by F_{i-1} .
- Drawback: over-cautions.

- 3. Each process is assigned a unique priority number. The priority numbers decide whether process P_i should wait for process P_j and therefore break the non-preemption condition.
- E.g. Assume that the philosophers' priorities are based on their ids, i.e., P_i has a higher priority than P_j if $i < j$. In this case P_i is allowed to wait for P_{i+1} for $i=1,2,3,4$. P_5 is not allowed to wait for P_1 . If this case happens, P_5 has to abort by releasing its acquired fork(s) (if any).
- Drawback: starvation. The lower priority one may always be rolled back. Solution is to raise the priority every time it is victimized.

- 4. Practically it is impossible to provide a method to break the mutual exclusion condition since most resources are intrinsically non-sharable, e.g., two philosophers cannot use the same fork at the same time.

A Deadlock Prevention Example

Wait-die

- | | | |
|------------------|--------|---------------|
| • Wants Resource | | Hold Resource |
| • Old process | -----→ | Young process |
| • 10 | | 20 |
| • Waits | | |

- Wants resource
 - Young process 20
 - Dies
- | | |
|--|----------------|
| | Holds resource |
| | Old process 10 |
- Wait-die is a non-preemptive method.

- **Wound-wait**

- Wants resource

Hold resource

- Old process 10

Young process 20

- Preempts

- Wants resource

Hold resource

- Young process 20

Old process 10

- Waits

An example

Process id	priority	1 st request time	length	Retry interval
P1	2	1	1	1
P2	1	1.5	2	1
P3	4	2.1	2	2
P4	5	3.3	1	1
P5	3	4.0	2	3

Deadlock Avoidance

Four resources ABCD. A has 6 instances, B has 3 instances, C Has 4 instances and D has 2 instances.

Process	Allocation	Max
	ABCD	ABCD
P1	3011	4111
P2	0100	0212
P3	1110	4210
P4	1101	1101
P5	0000	2110

Is the current state safe?

If P5 requests for (1,0,1,0), can this be granted?

Deadlock Detection and Recovery

- Centralized approaches
- Distributed approaches
- Hierarchical approaches